Using Atmosic Sensor Hub

Application Note

SUMMARY: This application note describes the settings, functionality, and code flow of the (Atmosic Sensor Hub) ATM_shub example code running on an Atmosic ATM2, ATM3, or ATM33/e Wireless SoC.

CONFIDENTIAL



Atmosic™

Table of Contents

Acronyms and Abbreviations **4**

1. Overview **5**

1.1 Quick Start 5

2. Building Application **11**

2.1 Devices 11

2.1.1 Customization Between BOARD Configurations 12

3. State Machines **12**

3.1 State Machine Flow Chart 13

3.2 Bluetooth Parameters Configuration 14

4. Sensor Configurations and Alarms **15**

4.1 Externally Connected Temperature Sensor 15

4.2 On Board Sensors on the EVK 15

4.2.1 EVK Rework 18

4.3 Sensor Hub Alarms 19

5. Power Management **23**

5.1 Differences Between ATM2/ATM3 vs. ATM33/e Series 23

5.1.1 ATM2/ATM3 Series 23

5.1.2 ATM33/e Series 23

6. Hardware Setup **24**

6.1 Pin Setup 24

6.1.1 ATM2/ATM3 Series 24

6.1.2 ATM33/e Series 25

6.2 Configure Memory Layout 26

6.2.1 Differences Between ATM2/ATM3 vs. ATM33/e Series 27

ATM2/ATM3 Series 27

ATM33/e Series 28

Reference Documents **32**

Revision History **33**

CONFIDENTIAL

List of Figures

Figure 1 - Cold Booting
Figure 2 - Initial Advertisement of Memory Before Data Collection
Figure 3 - Wake Up from Hibernation
Figure 4 - Advertisement of Collected Data
Figure 5 - State Machines for the SHUB
Figure 6 - State Transition Diagram
Figure 7 - Sample Console Logs of the Accelerometer Sensor
Figure 8 - Sample Console Logs of the Temperature Sensor
Figure 9 - ATMEVK_3330e_QN on Board Sensor Rework
Figure 10 - Sample Console Logs During Power On Reset
Figure 11 - Sample Console Logs Polled After Hibernation Cycle
Figure 12 - Sample Console Logs After an Alarm Event
Figure 13 - SRAM Bank Power Management
Figure 14 - Memory Layout and Access for ATM2/ATM3
Figure 15 - Allocation of Memory for ATM33/e
Figure 16 - SHUB OVERRIDE Power Configuration
Figure 17 - Memory Layout and Access for ATM33/e

List of Tables

Table 1 - Applicable Evaluation Kits
Table 2 - Device Build Options
Table 3 - Sensor Hub State Machines
Table 4 - Pinmap Settings for ATM2/ATM3 Series
Table 5 - Pinmap Settings for ATM33 Series

CONFIDENTIAL

Acronyms and Abbreviations

Acronyms	Definition
ATM2	ATM2201 ATM2202 ATM2221 ATM2231 ATM2251
ATM3	ATM3201 ATM3202 ATM3221 ATM3231
ATM33	ATM3330 ATM3325
ATM33e	ATM3330e
ATM33/e	ATM33/ATM33e
EVB	Evaluation Board
EVK	Evaluation Kit
I2C	Inter-Integrated Circuit
NVDS	Non-Volatile Data Storage
NVM	Non-Volatile Memory
SDK	Software Development Kit
SHUB	Sensor Hub
SoC	System-on-Chip
SPI	Serial Peripheral Interface
SRAM	Static Random Access Memory
PSEQ	Power Sequencer
PMU	Power Management Unit

CONFIDENTIAL

1. Overview

The Atmosic Sensor Hub (ATM_shub) example demonstrates how to use the sensor hub to poll connected sensors while the CPU is in hibernation. An external temperature sensor (referred to as PmodTMP2 in the rest of the document) is connected to the EVB using I2C pulled up by 10 KΩ resistors. It should be ensured that the PmodTMP2 has an operating voltage greater than 1.8 V (2.7 V to 5.5 V)

The sensor hub engine, through appropriate register configuration can read data from the sensors and store the resultant data into external flash (for ATM2/ATM3) or on volatile SRAM (for ATM33/e) when the SoC is in hibernation (or any other deep sleep mode). It does not need any assistance from the integrated CPU nor does it need any state to be retained. On a less frequent, periodic basis, both the CPU and Bluetooth LE subsystems will wake from hibernate to transmit the aggregated temperature data off chip over Bluetooth LE.

***Note**: When using ATM2/ATM3 devices, the application is expected to work only on Macronix external flash devices (OPCODE_PP set to 0x38 in shub.h) as specified in the README for the application.*

CONFIDENTIAL

1.1 Quick Start

- Prepare ATM2/ATM3 (which are connected with Atmosic I/O Board), or ATM33/e EVKs:

EVK	SoC Package	SoC Part Number	Kit Part Number
Evaluation Kit for ATM2202	40-pin 5x5 mm QFN	ATM2202SR	ATMEVK-M2202-02
Evaluation Kit for ATM2221	64-pin 6x6 mm QFN	ATM2221SR	ATMEVK-M2221-02
Evaluation Kit for ATM2251	37L WLCP	ATM2251SR	ATMEVK-M2251-01
Evaluation Kit for ATM3201	40-pin 5xt mm QFN	ATM3201SR	ATMEVK-M3201-02
Evaluation Kit for ATM3202	40-pin 5x5 mm QFN	ATM3202SR	ATMEVK-M3202-02
Evaluation Kit for ATM3221	64-pin 6x6 mm DR_QFN	ATM3221SR	ATMEVK-M3221-02
Evaluation Kit for ATM3325	40-pin 5x5 mm QFN	ATM3325-5DCAQK	ATMEVK-3325-QK
Evaluation Kit for ATM3325 with extended storage package	40-pin 5x5 mm QFN	ATM3325-5LCAQK	ATMEVK-3325-LQK

EVK	SoC Package	SoC Part Number	Kit Part Number
Evaluation Kit for ATM3330	56-pin 7x7 mm QFN	ATM3330-5DCAQN	ATMEVK-3330-QN
Evaluation Kit for ATM3330e	56-pin 7x7 mm QFN	ATM3330E-5DCAQN	ATMEVK-3330e-QN

Table 1 - Applicable Evaluation Kits

- Flash or NVM is required for this ATM_shub application in the case of ATM2/ATM3 Atmosic devices
- Install Atmosic SDK 5.4.0.1 or a later version
- Refer to the Pin Setup
- Go to the corresponding SDK directory location for the package type as inferred from Table 1, in the format of platform/<series>/<device-version>/example/ATM_shub folder of the Atmosic SDK
- Connect the PmodTMP2 external sensor based on the README file in the SDK example directory
- Type `make clean` then `make run_all BOARD=<board>` to program the sensor hub (see Pin Setup for more information)
- For ATM2/ATM3 the default external flash device is used to store the collected temperature data while in ATM33/e based systems, a section of data in the on-chip SRAM memory is allocated to store such data

Figure 1 shows the logs for a device waking up successfully from a cold boot:

CONFIDENTIAL

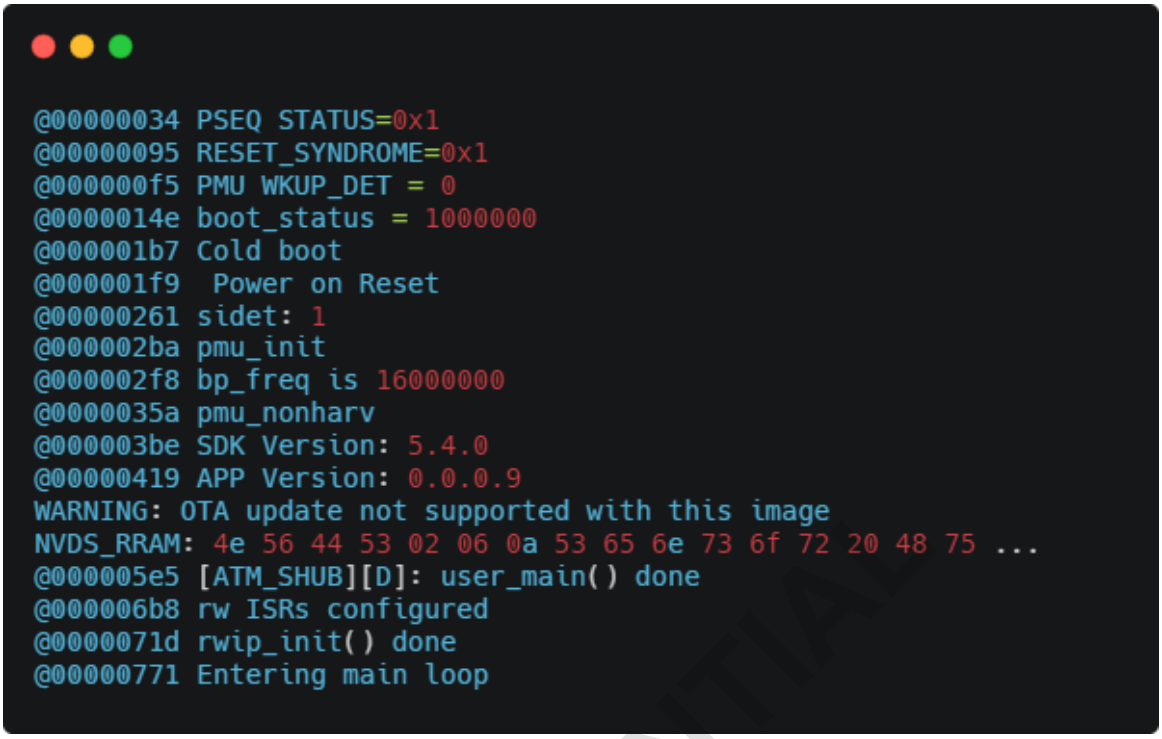


Figure 1 - Cold Booting

The SRAM (in the case of ATM33/e) is then cleared. Figure 2 shows the next step which is reading the existing data from the advertisement length which is to be advertised via the primary radio. Note the hexadecimal values correspond to the letters SHUB. This is used as an identifier for the start of the Sensor Hub data values in memory.

CONFIDENTIAL

```
@00000848 [atm_adv][D]: Adv dur 1(unit:10ms) max_adv_evt 0 (timeout lums)
@00000967 [ATM_SHUB][D]: adv_state = 2
@000009ff [atm_adv][D]: not valid format to append vendor app info
@00000b04 [ATM_SHUB][D]: adv_state = 4
@00000b9e Set PSEQ overrides
@00000bf8 setting loop_time to: fa00
@00000c6b Setting up SHUB0. mode=I2C, num_sensors=1
@00000d0c [ATM_SHUB][D]: shub sensor data:
@00000dac [ATM_SHUB][D]: 1c
@00000e20 [ATM_SHUB][D]: ff
@00000e95 [ATM_SHUB][D]: 53
@00000f09 [ATM_SHUB][D]: 48
@00000f7d [ATM_SHUB][D]: 55
@00000ff2 [ATM_SHUB][D]: 42
@00001066 [ATM_SHUB][D]: 00
@000010da [ATM_SHUB][D]: 00
@0000114f [ATM_SHUB][D]: 00
@000011c3 [ATM_SHUB][D]: 00
@00001237 [ATM_SHUB][D]: 00
@000012ac [ATM_SHUB][D]: 00
@00001320 [ATM_SHUB][D]: 00
@00001395 [ATM_SHUB][D]: 00
@00001409 [ATM_SHUB][D]: 00
@0000147e [ATM_SHUB][D]: 00
@000014f2 [ATM_SHUB][D]: 00
@00001566 [ATM_SHUB][D]: 00
@000015db [ATM_SHUB][D]: 00
@0000164f [ATM_SHUB][D]: 00
@000016c3 [ATM_SHUB][D]: 00
@00001738 [ATM_SHUB][D]: 00
@000017ac [ATM_SHUB][D]: 00
@00001820 [ATM_SHUB][D]: 00
@00001895 [ATM_SHUB][D]: 00
@00001909 [ATM_SHUB][D]: 00
@0000197e [ATM_SHUB][D]: 00
@000019f2 [ATM_SHUB][D]: 00
@00001a79 [ATM_SHUB][D]: adv_state = 6
@00001b34 [atm_adv][D]: Adv0: ON
@00001bba [ATM_SHUB][D]: adv_state = 9
@00001c5a [atm_adv][D]: Adv0: OFF (0x406)
@00001cfa [ATM_SHUB][D]: adv_state = 7
```

CONFIDENTIAL

Figure 2 - Initial Advertisement of Memory Before Data Collection

The '*adv_state = 6*' in the console log in Figure 2 indicates that the sensor data is ready for advertisement as denoted by the ATM_SHUB_S_ADV_STARTING state in the state machine in State Machine Flow Chart. (Note in the first cycle before hibernation, the collected data is empty hence 00 are seen in the console log) The `adv_state = 9` in the console log indicates the actual start of the advertisement of the

data as denoted by ATM_SHUB_S_ADV_STARTED in the state machine diagram. The `adv_state = 7` in the console log indicates that all the data has been advertised and that the advertisement has been stopped as denoted by ATM_SHUB_S_ADV_STOPPED in the state machine diagram. The SOC goes into hibernation after this state.

The SHUB engine wakes up and restarts (with operation mode ATM_SHUB_OP_RESTART_ADV) after it has collected a certain number of samples of data during the SOC hibernation cycle.

During the consecutive boot cycles after the initial cold boot, the reason for the boot is displayed as a Hibernation wakeup enabled by the `atm_pm_set_hib_restart_time(restart_time_csec);` API.

The SHUB engine then prints the value of the PmodTMP2 temperature sensor data which was collected over a period of 30 seconds, in 2 second intervals. A total of 15 samples of data are displayed on the terminal for the user to view. This data is also transmitted via the primary radio.

CONFIDENTIAL

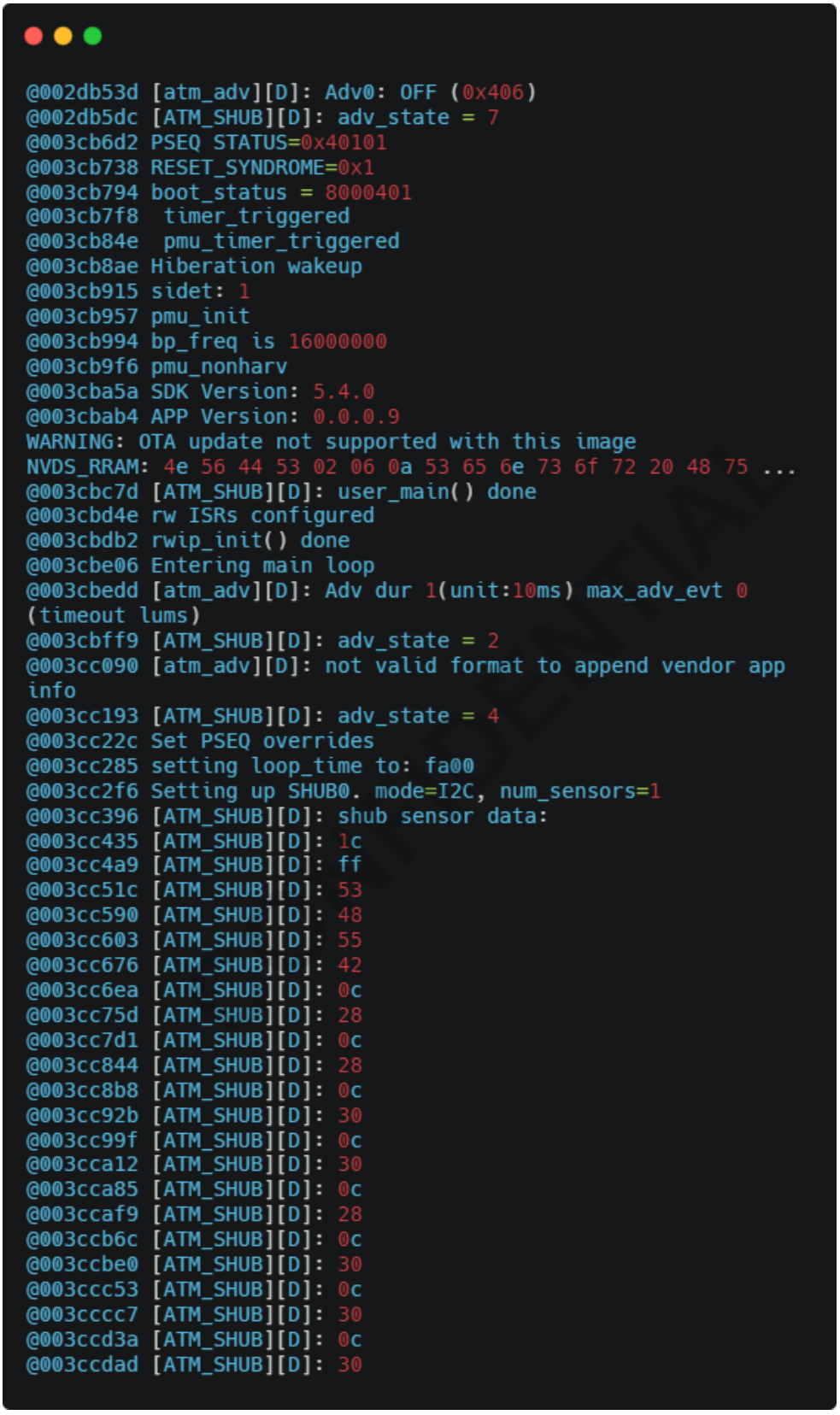


Figure 3 - Wake Up from Hibernation

The SHUB engine repeats this process continuously in cycles.



Figure 4 - Advertisement of Collected Data

2. Building Application

2.1 Devices

ATM2/ATM3 and ATM33/e series of Atmosic devices are supported by this application. The devices are selected through a BOARD variable from the command line. Table 2 describes build options of the devices.

Device	Description	Build Option
ATMx221	• 6x6 mm QFN package • External Flash	BOARD=ATMEVK_M2221 BOARD=ATMEVK_M3221
ATMx202	• 5x5 mm QFN package • Internal Flash	BOARD=ATMEVK_M2202 BOARD=ATMEVK_M3202
ATMx231	• 7x7 mm QFN package • External Flash	BOARD=ATMEVK_M2231 BOARD=ATMEVK_M3231
ATM3330	• 7x7 mm QFN package • Internal NVM	BOARD=ATMEVK_3330_QN
ATM3330e	• 7x7 mm QFN package • Energy Harvesting • Internal NVM	BOARD=ATMEVK_3330e_QN
ATM3325	• 5x5 mm package • Internal NVM	BOARD=ATMEVK_3325_QK
ATM3325_LQK	• 5x5 mm package • Internal NVM with extended storage	BOARD=ATMEVK_3325_LQK

Table 2 - Device Build Options

CONFIDENTIAL

If BOARD variable is not added in command line, ATMEVK_M2221 or ATMEVK_3330_QN option will be applied as the default when the location folder is in atm2 or atm33 respectively.

2.1.1 Customization Between BOARD Configurations

The current SDK example supports external Macronix flash devices to store the collected data on an ATM2/ATM3 configuration. On ATM33/e EVKs, which do not have an external flash, the data is stored in the SRAM region.

3. State Machines

State machines are the core of the application. Users can easily change and optimize design by understanding the state machines. State machines are created by defining state transition table mechanisms provided by the atm_asm module.

MMI State Name	Description
ATM_SHUB_S_INIT	The initial state. Marks the device wake up from cold boot.
ATM_SHUB_S_IDLE	Initialization in progress.
ATM_SHUB_S_ADV_STARTING	The end state (when adv is stopped then enters hibernate) or intermediate state (Initialized after a cold boot).
ATM_SHUB_S_ADV_STARTED	Broadcasts collected data packets in this state.
ATM_SHUB_S_ADV_STOPPED	Marks the end of the advertisement of data. Device is ready to enter hibernate to repeat the cycle.

Table 3 - Sensor Hub State Machines

CONFIDENTIAL

3.1 State Machine Flow Chart

```
/**
     * @brief ATM_SHUB_sensor state machine step by step
     *
     * Step 1: Gap init
     * Step 2: Create adv
     * Step 3: Start adv
     * Step 4: Adv timeout, stop adv and hibernate
     * Step 5: Adv timeout, restart adv
     */
    static state_entry const s_tbl[] = {
        {S_OP(ATM_SHUB_S_INIT, ATM_SHUB_OP_MODULE_INIT),
ATM_SHUB_S_IDLE,
            atm_shub_adv_init},
        {S_OP(ATM_SHUB_S_IDLE, ATM_SHUB_OP_START_ADV),
ATM_SHUB_S_ADV_STARTING,
            atm_shub_adv_start_adv},
        {S_OP(ATM_SHUB_S_ADV_STARTING, ATM_SHUB_OP_START_ADV_CFM),
            ATM_SHUB_S_ADV_STARTED, atm_shub_adv_start_on},
        {S_OP(ATM_SHUB_S_ADV_STARTED, ATM_SHUB_OP_ADV_TIMEOUT),
            ATM_SHUB_S_ADV_STOPPED, atm_shub_adv_timeout},
        {S_OP(ATM_SHUB_S_ADV_STOPPED, ATM_SHUB_OP_RESTART_ADV),
            ATM_SHUB_S_ADV_STARTING, atm_shub_adv_restart_adv},
    };
```

Figure 5 - State Machines for the SHUB

CONFIDENTIAL

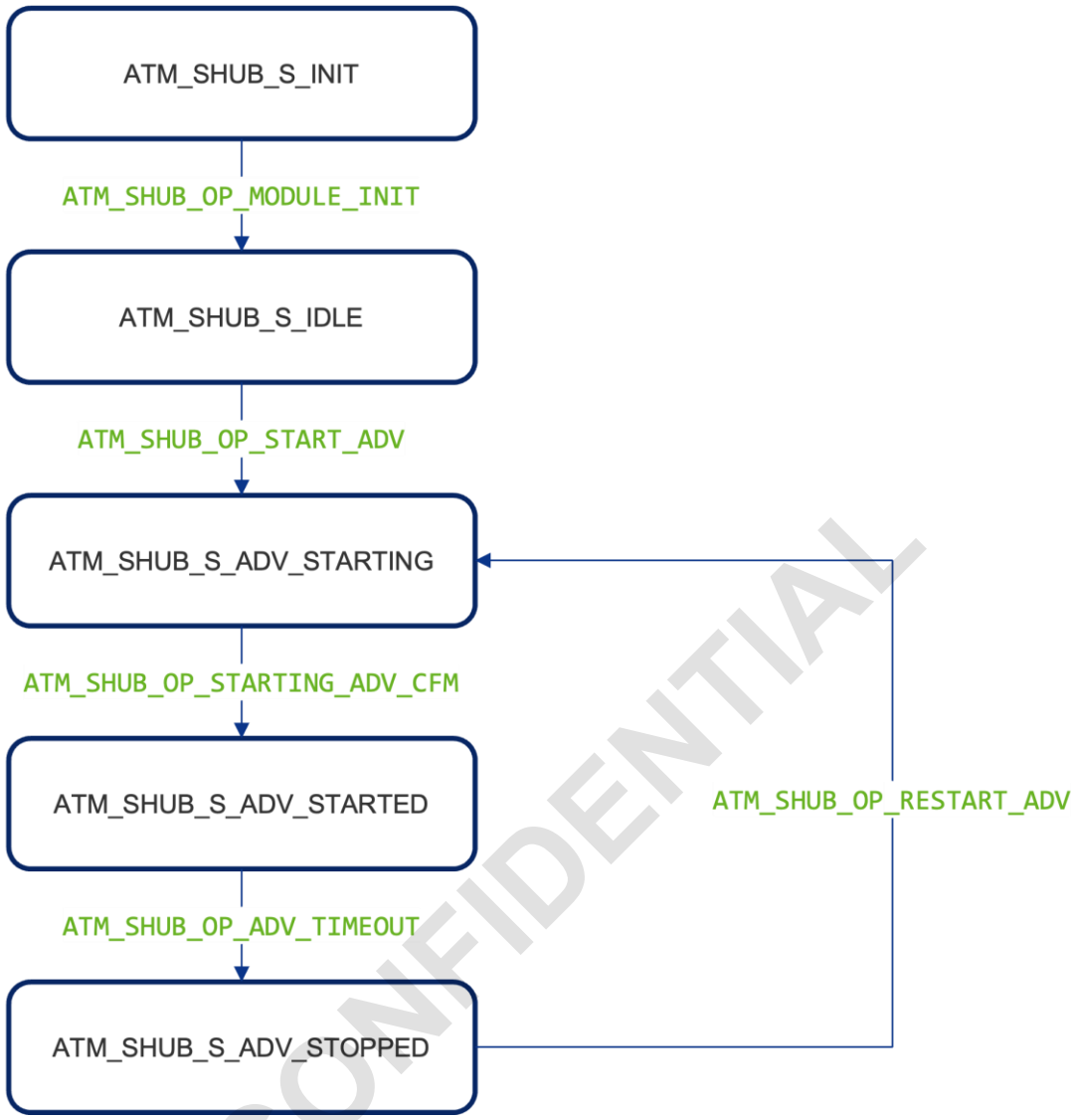


Figure 6 - State Transition Diagram

CONFIDENTIAL

The Sensor hub example wakes up and collects data and broadcasts the collected data without entering the CONNECTED state. A paired connection is not necessary to run the sensor hub example. After the ATM_SHUB_S_ADV_STOPPED state, the system enters a hibernation state. The restart adv timeout then kicks off and wakes up the system from hibernation state.

3.2 Bluetooth Parameters Configuration

In this application, variable features and configurations are supported. The bluetooth parameter configurations are defined in atm_shub_adv_params.h

4. Sensor Configurations and Alarms

For ease of demonstration, the Sensor Hub has an option to poll and collect data from either of the on board temperature sensors (temperature sensor - ENS210 or the accelerometer - LISXDH) or connect an external temperature sensor (PmodTMP2) The current ATM_shub example only supports polling the on-board sensors on the Paris versions of the EVKs. When polling the on-board temperature sensor (ENS210), the sensor hub application also turns on monitoring alarms using the Sensor Hub hardware by default.

4.1 Externally Connected Temperature Sensor

The default externally connected sensor that works with the Sensor Hub is the PmodTMP2 sensor. This works on both ATM2/ATM3 devices as mentioned in the 1.1 Quick Start section.

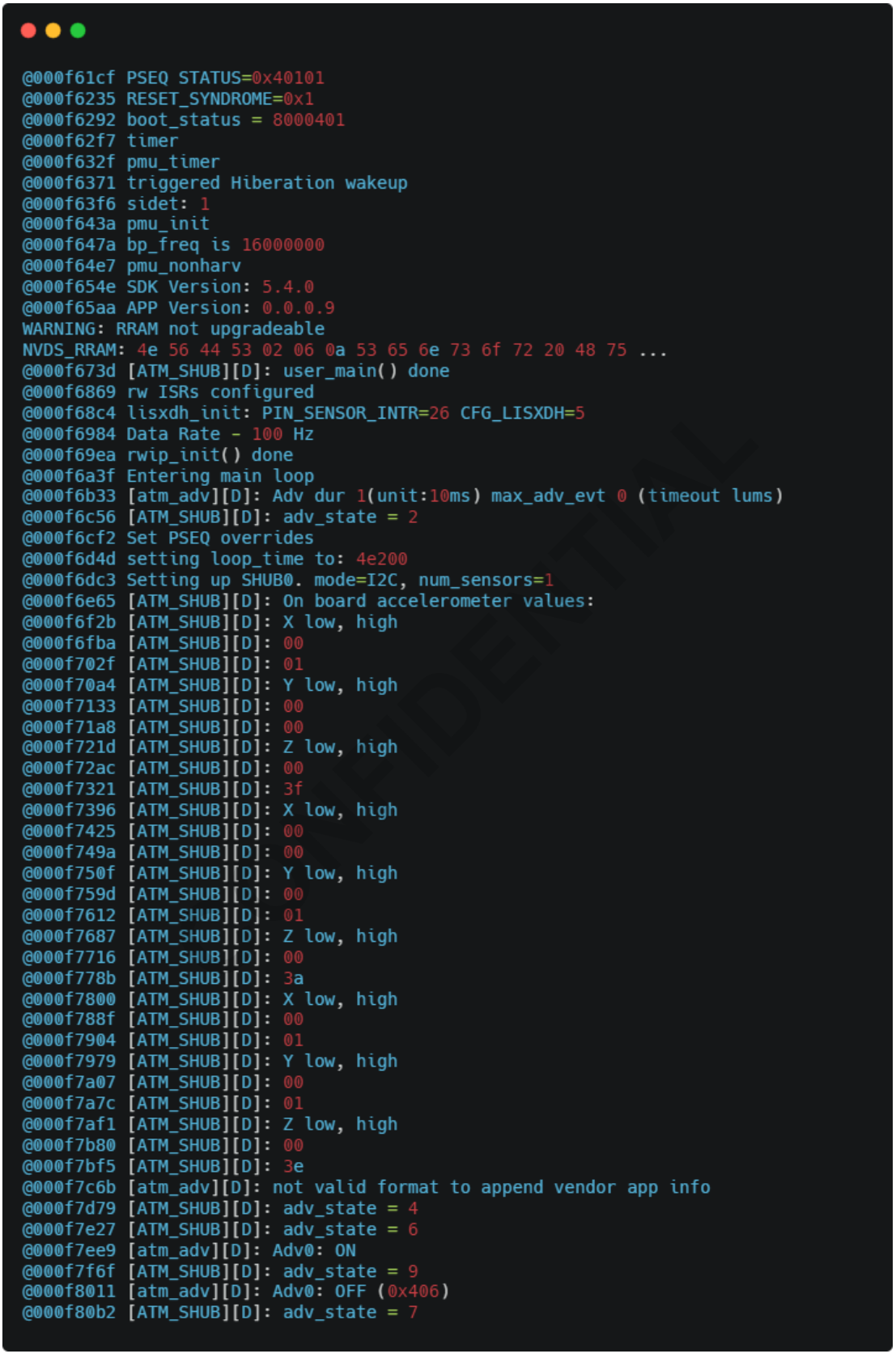
4.2 On Board Sensors on the EVK

There are two on-board sensors which exist on the ATM2/3 EVKs which are the temperature/ humidity sensor (ENS210) and the accelerometer sensor (LISXDH)

In the existing ATM_shub example, the on board sensor is only supported on the ATM3 versions of the EVK. To enable the on board sensors, the jumper connecting both the on board sensors (temperature sensor and accelerometer) should be powered on by connecting the Jumper pin (JP25 on the ATMEVK-3330E-QN, 7x7 mm package) to power the on board sensors. Either of the on board sensors can be enabled by passing in CFG_ENS210_SENSOR=1 for enabling the on board temperature sensor or CFG_LISXDH_SENSOR=1 for enabling the on board accelerometer. More details on this are mentioned in the README file in the ATM_shub example. Note: No more than one sensor can be enabled at the same time.

The collected accelerometer data would look as shown in the Figure 7 below where the X, Y and Z values correspond to the position and acceleration of the EVK.

CONFIDENTIAL



```
@000f61cf PSEQ STATUS=0x40101
@000f6235 RESET_SYNDROME=0x1
@000f6292 boot_status = 8000401
@000f62f7 timer
@000f632f pmu_timer
@000f6371 triggered Hiberation wakeup
@000f63f6 sidet: 1
@000f643a pmu_init
@000f647a bp_freq is 16000000
@000f64e7 pmu_nonharv
@000f654e SDK Version: 5.4.0
@000f65aa APP Version: 0.0.0.9
WARNING: RRAM not upgradeable
NVDS_RRAM: 4e 56 44 53 02 06 0a 53 65 6e 73 6f 72 20 48 75 ...
@000f673d [ATM_SHUB][D]: user_main() done
@000f6869 rw ISRs configured
@000f68c4 lisxdh_init: PIN_SENSOR_INTR=26 CFG_LISXDH=5
@000f6984 Data Rate - 100 Hz
@000f69ea rwip_init() done
@000f6a3f Entering main loop
@000f6b33 [atm_adv][D]: Adv dur 1(unit:10ms) max_adv_evt 0 (timeout lums)
@000f6c56 [ATM_SHUB][D]: adv_state = 2
@000f6cf2 Set PSEQ overrides
@000f6d4d setting loop_time to: 4e200
@000f6dc3 Setting up SHUB0. mode=I2C, num_sensors=1
@000f6e65 [ATM_SHUB][D]: On board accelerometer values:
@000f6f2b [ATM_SHUB][D]: X low, high
@000f6fba [ATM_SHUB][D]: 00
@000f702f [ATM_SHUB][D]: 01
@000f70a4 [ATM_SHUB][D]: Y low, high
@000f7133 [ATM_SHUB][D]: 00
@000f71a8 [ATM_SHUB][D]: 00
@000f721d [ATM_SHUB][D]: Z low, high
@000f72ac [ATM_SHUB][D]: 00
@000f7321 [ATM_SHUB][D]: 3f
@000f7396 [ATM_SHUB][D]: X low, high
@000f7425 [ATM_SHUB][D]: 00
@000f749a [ATM_SHUB][D]: 00
@000f750f [ATM_SHUB][D]: Y low, high
@000f759d [ATM_SHUB][D]: 00
@000f7612 [ATM_SHUB][D]: 01
@000f7687 [ATM_SHUB][D]: Z low, high
@000f7716 [ATM_SHUB][D]: 00
@000f778b [ATM_SHUB][D]: 3a
@000f7800 [ATM_SHUB][D]: X low, high
@000f788f [ATM_SHUB][D]: 00
@000f7904 [ATM_SHUB][D]: 01
@000f7979 [ATM_SHUB][D]: Y low, high
@000f7a07 [ATM_SHUB][D]: 00
@000f7a7c [ATM_SHUB][D]: 01
@000f7af1 [ATM_SHUB][D]: Z low, high
@000f7b80 [ATM_SHUB][D]: 00
@000f7bf5 [ATM_SHUB][D]: 3e
@000f7c6b [atm_adv][D]: not valid format to append vendor app info
@000f7d79 [ATM_SHUB][D]: adv_state = 4
@000f7e27 [ATM_SHUB][D]: adv_state = 6
@000f7ee9 [atm_adv][D]: Adv0: ON
@000f7f6f [ATM_SHUB][D]: adv_state = 9
@000f8011 [atm_adv][D]: Adv0: OFF (0x406)
@000f80b2 [ATM_SHUB][D]: adv_state = 7
```

Figure 7 - Sample Console Logs of the Accelerometer Sensor

Figure 8 shows a sample of the output produced by the on board temperature sensor.

```
@001e71e9 PSEQ STATUS=0x40101
@001e724e RESET_SYNDROME=0x1
@001e72ab boot_status = 8000401
@001e7311 timer
@001e7348 pmu_timer
@001e7389 triggered Hiberation wakeup
@001e740f sidet: 1
@001e7453 pmu_init
@001e7492 bp_freq is 16000000
@001e7500 pmu_nonharv
@001e7566 SDK Version: 5.4.0
@001e75c2 APP Version: 0.0.0.9
WARNING: RRAM not upgradeable
NVDS_RRAM: 4e 56 44 53 02 06 0a 53 65 6e 73 6f 72 20 48 75 ...
@001e7755 [ATM_SHUB][D]: user_main() done
@001e7880 rw ISRs configured
@001e78e5 rwip_init() done
@001e793a Entering main loop
@001e7a2f [atm_adv][D]: Adv dur 1(unit:10ms) max_adv_evt 0 (timeout lums)
@001e7b53 [ATM_SHUB][D]: adv_state = 2
@001e7bef Set PSEQ overrides
@001e7c4a setting loop_time to: fa00
@001e7cbd Setting up SHUB0. mode=I2C, num_sensors=1
@001e7d5e [ATM_SHUB][D]: On board temperature values:
@001e7e20 [ATM_SHUB][D]: 26 deg C
@001e7ea8 [ATM_SHUB][D]: 26 deg C
@001e7f2f [ATM_SHUB][D]: 26 deg C
@001e7fb7 [ATM_SHUB][D]: 26 deg C
@001e803f [ATM_SHUB][D]: 26 deg C
@001e80c7 [ATM_SHUB][D]: 26 deg C
@001e814e [ATM_SHUB][D]: 26 deg C
@001e81d6 [ATM_SHUB][D]: 26 deg C
@001e825e [ATM_SHUB][D]: 26 deg C
@001e82e5 [ATM_SHUB][D]: 26 deg C
@001e836d [ATM_SHUB][D]: 26 deg C
@001e83f5 [atm_adv][D]: not valid format to append vendor app info
@001e8503 [ATM_SHUB][D]: adv_state = 4
@001e85b1 [ATM_SHUB][D]: adv_state = 6
@001e8672 [atm_adv][D]: Adv0: ON
@001e86f8 [ATM_SHUB][D]: adv_state = 9
@001e879a [atm_adv][D]: Adv0: OFF (0x406)
@001e883b [ATM_SHUB][D]: adv_state = 7
```

Figure 8 - Sample Console Logs of the Temperature Sensor

***Note**: The default EVK connections should be re-worked to share the I2C lines and the SHUB_I2C GPIOs as described in EVK Rework below.*

4.2.1 EVK Rework

In order to facilitate the polling of the on board sensors using the sensor hub, some changes need to be made to the existing EVB. Depending on the board, the 7x7 mm package EVBs need a small rework to hook up some SHUB I2C lines. On the latest 7x7 mm package EVB, the rework for SHUB can be as simple as adding two wires connecting P4 to P6 and P5 to P7 on the 50 pin header.

The implications on the latest 7x7 mm package EVB (ATMEVK-3330e-QN, ATMEVK-3330-QN) of this would be:

1) In software, P6 and P7 will have to be put in hi-Z.
2) Button_1 will not be functional (Button_2 will still be good) because it is currently connected to P5
3) Li-ion charging circuit will not be functional because P4 is currently used as the 32 kHz clock driving the booster circuit.

The implications on the latest 5x5 mm package EVB (ATMEVK-3325-QK) of this would be:

1) In software, P6 and P7 will have to be put in hi-Z.
2) Button_1 will not be functional (Button_2 will still be functional) because it is currently connected to P5
3) LED_2 will not be functional because it is currently driven by P5.

On the 5x5 mm with Extended Storage package EVBs (ATMEVK-3325-LQK), this should have been put on P4/5 already, so all that is needed is a jumper on the relevant jumper header.

See Figure 9 for the rework performed on an ATMEVK_3330e_QN EVB to facilitate using on board sensors.

CONFIDENTIAL

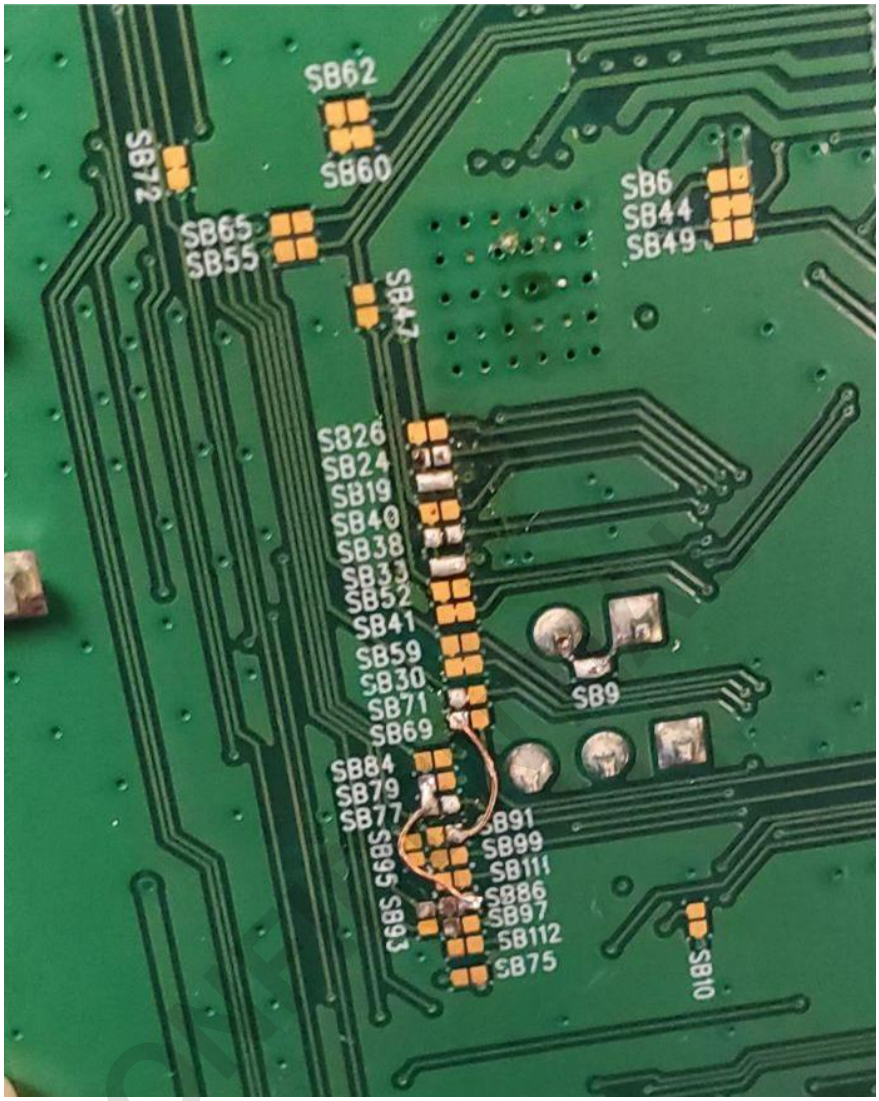


Figure 9 - ATMEVK_3330e_QN on Board Sensor Rework

4.3 Sensor Hub Alarms

On the ATM3 EVK series, the ATM_shub example also provides additional support for an event triggered wake up from hibernation. The sensor hub wakes up from the hibernation state if the polled values from the sensor are above or below a certain threshold. For ease of use, this ‘Alarm’ feature of the sensor hub is turned ON by default when using the on board temperature sensor. The temperature thresholds are calculated on the fly during power-on reset with a 10% variance based on the temperature read during power on reset. More details on how the thresholds are calculated are mentioned in the README file in the ATM_shub example.

Additional details on how the alarm registers are set up, refer to the **Using ATM2/ATM3 Sensor Hub White Paper**.

See Figure 10 for sample console logs when the on board temperature sensor is polled during power on reset:

```
@00000142 boot_status = 1000000
@000001a9 Cold boot
@000001ea  Power on Reset
@00000250 sidet: 1
@000002ab pmu_init
@000002ec bp_freq is 16000000
@0000035d pmu_nonharv
@000003c5 SDK Version: 5.4.0
@00000424 APP Version: 0.0.0.9
WARNING: RRAM not upgradeable
NVDS_RRAM: 4e 56 44 53 02 06 0a 53 65 6e 73 6f 72 20 48 75 ...
@000005c2 [ATM_SHUB][D]: user_main() done
@000006f5 rw ISRs configured
@000007a8 On board temperature 23 degC VARIANCE 2 degC
@00000856 Min threshold 21 degC Max threshold 25 degC
@00000908 rwip_init() done
@00000960 Entering main loop
@00000a5b [atm_adv][D]: Adv dur 1(unit:10ms) max_adv_evt 0 (timeout lums)
@00000b86 [ATM_SHUB][D]: adv_state = 2
@00000c26 Set PSEQ overrides
@00000c84 setting loop_time to: fa00
@00000cfa Setting up SHUB0. mode=I2C, num_sensors=1
@00000da0 [ATM_SHUB][D]: On board temperature values:
@00000e66 [ATM_SHUB][D]: 0 deg C
@00000eee [ATM_SHUB][D]: 0 deg C
@00000f76 [ATM_SHUB][D]: 0 deg C
@00000ffe [ATM_SHUB][D]: 0 deg C
@00001085 [ATM_SHUB][D]: 0 deg C
@0000110d [ATM_SHUB][D]: 0 deg C
@00001195 [ATM_SHUB][D]: 0 deg C
@0000121d [ATM_SHUB][D]: 0 deg C
@000012a4 [ATM_SHUB][D]: 0 deg C
@0000132c [ATM_SHUB][D]: 0 deg C
@000013b3 [ATM_SHUB][D]: 0 deg C
@0000143c [atm_adv][D]: not valid format to append vendor app info
@00001551 [ATM_SHUB][D]: adv_state = 4
@00001604 [ATM_SHUB][D]: adv_state = 6
@000016cb [atm_adv][D]: Adv0: ON
@00001754 [ATM_SHUB][D]: adv_state = 9
@000017fb [atm_adv][D]: Adv0: OFF (0x406)
@000018a0 [ATM_SHUB][D]: adv_state = 7
```

Figure 10 - Sample Console Logs During Power On Reset

The reason for the reset is ‘Cold boot, Power on Reset’. The current on board sensor is read to be 23 degrees Celcius and the minimum and maximum temperature thresholds are calculated with a 10% variance as described in the README file. The initial

temperature values are read out as 0 degree celsius since the data would only be collected during a hibernation cycle of the Sensor Hub.

After the first hibernation cycle:

```
@002d8932 PSEQ STATUS=0x40101
@002d8997 RESET_SYNDROME=0x1
@002d89f4 boot_status = 8000401
@002d8a5a timer
@002d8a91 pmu_timer
@002d8ad2 triggered Hiberation wakeup
@002d8b58 sidet: 1
@002d8b9c pmu_init
@002d8bdb bp_freq is 16000000
@002d8c48 pmu_nonharv
@002d8caf SDK Version: 5.4.0
@002d8d0b APP Version: 0.0.0.9
WARNING: RRAM not upgradeable
NVDS_RRAM: 4e 56 44 53 02 06 0a 53 65 6e 73 6f 72 20 48 75 ...
@002d8e9d [ATM_SHUB][D]: user_main() done
@002d8fc8 rw ISRs configured
@002d902d rwip_init() done
@002d9082 Entering main loop
@002d9176 [atm_adv][D]: Adv dur 1(unit:10ms) max_adv_evt 0 (timeout lums)
@002d929a [ATM_SHUB][D]: adv_state = 2
@002d9336 Set PSEQ overrides
@002d9391 setting loop_time to: fa00
@002d9403 Setting up SHUB0. mode=I2C, num_sensors=1
@002d94a5 [ATM_SHUB][D]: On board temperature values:
@002d9567 [ATM_SHUB][D]: 23 deg C
@002d95ef [ATM_SHUB][D]: 23 deg C
@002d9676 [ATM_SHUB][D]: 23 deg C
@002d96fe [ATM_SHUB][D]: 23 deg C
@002d9786 [ATM_SHUB][D]: 23 deg C
@002d980e [ATM_SHUB][D]: 23 deg C
@002d9895 [ATM_SHUB][D]: 23 deg C
@002d991d [ATM_SHUB][D]: 23 deg C
@002d99a4 [ATM_SHUB][D]: 23 deg C
@002d9a2d [ATM_SHUB][D]: 23 deg C
@002d9ab4 [ATM_SHUB][D]: 23 deg C
@002d9b3c [atm_adv][D]: not valid format to append vendor app info
@002d9c4a [ATM_SHUB][D]: adv_state = 4
@002d9cf8 [ATM_SHUB][D]: adv_state = 6
@002d9dba [atm_adv][D]: Adv0: ON
@002d9e3f [ATM_SHUB][D]: adv_state = 9
@002d9ee1 [atm_adv][D]: Adv0: OFF (0x406)
@002d9f82 [ATM_SHUB][D]: adv_state = 7
```

Figure 11 - Sample Console Logs Polled After Hibernation Cycle

Notice how the restart reason is pmu_timer triggered. The minimum and maximum alarm temperature thresholds which were calculated during power on reset would still remain the same and would not change between hibernation cycles.

After an alarm trigger event:

When the temperature collected is out of bounds of the minimum / maximum threshold, the Sensor Hub engine immediately wakes up the SoC from hibernation and transmits the advertisement packet. The last valid temperature value collected by the Sensor hub engine would be the temperature which was out of bounds from the expected threshold.

Figure 12 shows the sample console logs when the on board temperature sensor is polled after an alarm event.

```
@03cd0293 PSEQ STATUS=0x48001
@03cd02f8 RESET_SYNDROME=0x1
@03cd0355 boot_status = 8000480
@03cd03bd shub
@03cd03ef pmu_timer
@03cd0430 triggered Hiberation wakeup
@03cd04b6 sidet: 1
@03cd04fa pmu_init
@03cd0539 bp_freq is 16000000
@03cd05a6 pmu_nonharv
@03cd060c SDK Version: 5.4.0
@03cd0668 APP Version: 0.0.0.9
WARNING: RRAM not upgradeable
NVDS_RRAM: 4e 56 44 53 02 06 0a 53 65 6e 73 6f 72 20 48 75 ...
@03cd07fb [ATM_SHUB][D]: user_main() done
@03cd0926 rw ISRs configured
@03cd0984 SHUB TRIGGERED ALARM
@03cd09ea rwip_init() done
@03cd0a40 Entering main loop
@03cd0b34 [atm_adv][D]: Adv dur 0(unit:10ms) max_adv_evt 0 (timeout lums)
@03cd0c58 [ATM_SHUB][D]: adv_state = 2
@03cd0cf3 Set PSEQ overrides
@03cd0d4e setting loop_time to: fa00
@03cd0dc1 Setting up SHUB0. mode=I2C, num_sensors=1
@03cd0e63 [ATM_SHUB][D]: On board temperature values:
@03cd0f24 [ATM_SHUB][D]: 24 deg C
@03cd0fac [ATM_SHUB][D]: 24 deg C
@03cd1034 [ATM_SHUB][D]: 24 deg C
@03cd10bc [ATM_SHUB][D]: 24 deg C
@03cd1144 [ATM_SHUB][D]: 24 deg C
@03cd11cb [ATM_SHUB][D]: 24 deg C
@03cd1253 [ATM_SHUB][D]: 24 deg C
@03cd12da [ATM_SHUB][D]: 29 deg C
@03cd1361 [ATM_SHUB][D]: 0 deg C
@03cd13e5 [ATM_SHUB][D]: 0 deg C
@03cd1469 [ATM_SHUB][D]: 0 deg C
@03cd14ee [atm_adv][D]: not valid format to append vendor app info
@03cd15fc [ATM_SHUB][D]: adv_state = 4
@03cd16aa [ATM_SHUB][D]: adv_state = 6
@03cd176b [atm_adv][D]: Adv0: ON
@03cd17f1 [ATM_SHUB][D]: adv_state = 9
```

Figure 12 - Sample Console Logs After an Alarm Event

In Figure 12 above the reason for wakeup from hibernation is `shub` instead of `pmu_timer` as in the previous case.

The ATM_shub continuously advertises the last packet containing the temperature data which was out of bounds when an alarm was triggered. This is done to ensure that the end user does not miss an actual alarm trigger due to the lack of continuous advertisements. The original state machine described in Section 3 would be paused and the machine would simply continuously advertise the last known packet when an alarm is triggered. In the above figure, the faulting temperature was 29 degrees Celcius which was also the last collected temperature value which was out of the expected bounds.

5. Power Management

The power management using the PSEQ is different for different hardware configurations on our EVKs. This is because based on whether the external flash memory is available, the way the SHUB engine stores the data is different.

5.1 Differences Between ATM2/ATM3 vs. ATM33/e Series

5.1.1 ATM2/ATM3 Series

ATM2/ATM3 series of Atmosic devices support non-volatile (external) flash memory by default. Since this memory is non-volatile, the memory does not need to stay powered ON during hibernation to retain the sensor samples collected by the SHUB engine. Hence, no changes in terms of power management are required on the ATM2/ATM3 series of Atmosic devices to support the SHUB engine.

5.1.2 ATM33/e Series

On ATM33/e based systems where SHUB data is stored in on-chip SRAM, the PMU should be configured to keep the SRAM powered on during hibernation as well. PSEQ OVERRIDES are configured based on the `UNINIT` memory section allocated on SRAM to store the SHUB data. Specific details on the `UNINIT` memory section are explained in Configure Memory Layout.

The section in the PSEQ handling hibernation considers only those SRAM banks that store SHUB data to keep powered ON. This is done in an effort to save power and shut off the banks which do not have any data expected to be stored in them.

CONFIDENTIAL

```
 // Keep SRAM memory banks ON through hibernate SHUB needs it
    // Make sure SHUB is out of reset, head switch closed, not
isolated and
    // clock enabled
    uint32_t const volatile sensor_hub_control = CMSDK_PSEQ-
>SENSOR_HUB_CONTROL;
    if (!(sensor_hub_control &
(PSEQ_SENSOR_HUB_CONTROL__SHUB_ISO__MASK |
            PSEQ_SENSOR_HUB_CONTROL__SHUB_VDDCUT__MASK |
            PSEQ_SENSOR_HUB_CONTROL__SHUB_RESET__MASK)) &&
        (sensor_hub_control &
PSEQ_SENSOR_HUB_CONTROL__SHUB_CLKEN__MASK)) {
        bool use_sram;
        uintptr_t sram_start_addr;
        uintptr_t sram_end_addr;
        WRPR_CTRL_PUSH(CMSDK_SHUB, WRPR_CTRL__CLK_ENABLE)
        {
            sram_start_addr =
                (uintptr_t)(CMSDK_SHUB->FLASH_CTRL1 +
CMSDK_SRAM_BASE);
            sram_end_addr =
                (uintptr_t)(CMSDK_SHUB->FLASH_CTRL2 +
CMSDK_SRAM_BASE);
            use_sram = CMSDK_SHUB->SETUP0 &
SHUB_SETUP0__STORE_IN_SRAM__MASK;
```

Figure 13 - SRAM Bank Power Management

CONFIDENTIAL

6. Hardware Setup

The hardware setups between ATM2/ATM33 and ATM33/e differ not only in terms of the board layout and pin setups but also in the way the SHUB engine stores the collected sensor data in memory.

6.1 Pin Setup

The pin configurations of BOARD are defined in driver/pinmux/pinmap_<BOARD>.h by default. The ATM_shub overwrites the setting by adding pinmap/pinmap_<BOARD>_overlay.h file in the project folder.

6.1.1 ATM2/ATM3 Series

Table 4 lists the pin settings in makefile according to the BOARD value of the ATM2/ATM3 series of Atmosic devices.

pinmap_ATMEVK_M2251.h, pinmap_ATMEVK_M2231.h, pinmap_ATMEVK_M2201.h, pinmap_ATMEVK_M3231.h, pinmap_ATMEVK_M2202.h, pinmap_ATMEVK_M3201.h, pinmap_ATMEVK_M3202.h		
Module	**#define in pinmap header file**	**Description**
Sensor Hub	#define PIN_SHUB_I2C0_SCL 9 #define PIN_SHUB_I2C0_SDA 30 #define PIN_SHUB_I2C1_SCL 11 #define PIN_SHUB_I2C1_SDA 13 #define PIN_SHUB_SPI0_CSN 10 #define PIN_SHUB_SPI0_SCLK 20 #define PIN_SHUB_SPI0_MOSI 22 #define PIN_SHUB_SPI0_MISO 23 #define PIN_SHUB_SPI1_CSN 25 #define PIN_SHUB_SPI1_SCLK 24 #define PIN_SHUB_SPI1_MOSI 2 #define PIN_SHUB_SPI1_MISO 33	The port 0 and 1 pins for connecting I2C / SPI lines for the sensor hub
pinmap_ATMEVK_M3221.h, pinmap_x2xx_emu.h, pinmap_ATMEVK_M2221.h		
Module	**#define in pinmap header file**	**Description**
Sensor Hub	#define PIN_SHUB_I2C0_SCL 3 #define PIN_SHUB_I2C0_SDA 4 #define PIN_SHUB_I2C1_SCL 12 #define PIN_SHUB_I2C1_SDA 13 #define PIN_SHUB_SPI0_CSN 11 #define PIN_SHUB_SPI0_SCLK 10 #define PIN_SHUB_SPI0_MOSI 12 #define PIN_SHUB_SPI0_MISO 13 #define PIN_SHUB_SPI1_CSN 25 #define PIN_SHUB_SPI1_SCLK 24 #define PIN_SHUB_SPI1_MOSI 26 #define PIN_SHUB_SPI1_MISO 27	The port 0 and 1 pins for connecting I2C / SPI lines for the sensor hub

Table 4 - Pinmap Settings for ATM2/ATM3 Series

CONFIDENTIAL

6.1.2 ATM33/e Series

pinmap_ATMEVK_3330_QN.h, pinmap_ATMEVK_3330e_QN.h		
Module	**#define in pinmap header file**	**Description**
Sensor Hub	#define PIN_SHUB_I2C0_SCL 12 #define PIN_SHUB_I2C0_SDA 13 #define PIN_SHUB_I2C1_SCL 2 #define PIN_SHUB_I2C1_SDA 3 #define PIN_SHUB_SPI0_CSN 8 #define PIN_SHUB_SPI0_SCLK 17	The port 0 and 1 pins for connecting I2C / SPI lines for the sensor hub

	#define PIN_SHUB_SPI0_MOSI 28 #define PIN_SHUB_SPI0_MISO 29 #define PIN_SHUB_SPI1_CSN 4 #define PIN_SHUB_SPI1_SCLK 5 #define PIN_SHUB_SPI1_MOSI 6 #define PIN_SHUB_SPI1_MISO 7	
pinmap_ATMEVK_3325_LQK.h		
Module	**#define in pinmap header file**	**Description**
Sensor Hub	#define PIN_SHUB_I2C0_SCL 20 #define PIN_SHUB_I2C0_SDA 17 #define PIN_SHUB_I2C1_SCL 18 #define PIN_SHUB_I2C1_SDA 19 #define PIN_SHUB_SPI0_CSN 30 #define PIN_SHUB_SPI0_SCLK 1 #define PIN_SHUB_SPI0_MOSI 18 #define PIN_SHUB_SPI0_MISO 3 #define PIN_SHUB_SPI1_CSN 4 #define PIN_SHUB_SPI1_SCLK 5 #define PIN_SHUB_SPI1_MOSI 22 #define PIN_SHUB_SPI1_MISO 7	The port 0 and 1 pins for connecting I2C / SPI lines for the sensor hub
pinmap_ATMEVK_3325_QK.h, pinmap_ATMEVK_3325_TAG.h		
Module	**#define in pinmap header file**	**Description**
Sensor Hub	#define PIN_SHUB_I2C0_SCL 12 #define PIN_SHUB_I2C0_SDA 17 #define PIN_SHUB_I2C1_SCL 18 #define PIN_SHUB_I2C1_SDA 19 #define PIN_SHUB_SPI0_CSN 30 #define PIN_SHUB_SPI0_SCLK 1 #define PIN_SHUB_SPI0_MOSI 18 #define PIN_SHUB_SPI0_MISO 3 #define PIN_SHUB_SPI1_CSN 4 #define PIN_SHUB_SPI1_SCLK 5 #define PIN_SHUB_SPI1_MOSI 6 #define PIN_SHUB_SPI1_MISO 7	The port 0 and 1 pins for connecting I2C / SPI lines for the sensor hub

Table 5 - Pinmap Settings for ATM33 Series

CONFIDENTIAL

6.2 Configure Memory Layout

The memory layout across different hardware differs depending on where the data is stored - either in external flash memory or in on-chip volatile SRAM.

6.2.1 Differences Between ATM2/ATM3 vs. ATM33/e Series

The following details the differences between the memory layout and where the SHUB sensor data is collected from and stored between external flash memory (ATM2/ATM3 series) and the on-chip SRAM memory (ATM33/e series) The SHUB feature is enabled by default. The flash essentially refers to either the external flash or the on-chip SRAM depending on the hardware used.

ATM2/ATM3 Series

For ATM2/ATM3, the makefile in the ATM_shub example directory on the SDK contains the USER_FLASH variable which allocates a size on the external flash device on which the sensor data is stored. The size of the allocated flash is fixed to 4096 bytes.

```
USER_SIZE := 4096
```

During compile time, this memory gets statically allocated depending on the value of SHUB_DATA_SAMPLES the user expects to collect.

CONFIDENTIAL

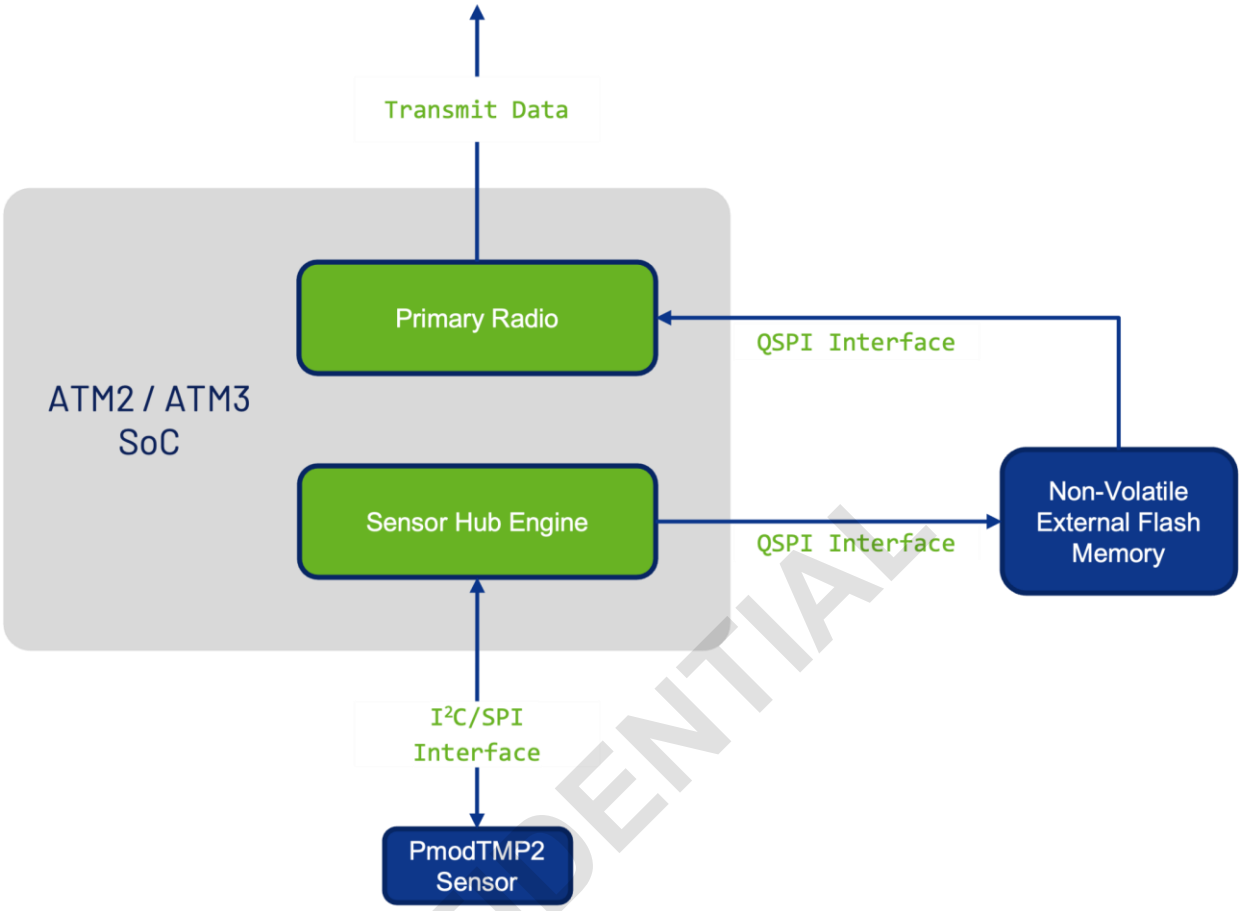


Figure 14 - Memory Layout and Access for ATM2/ATM3

CONFIDENTIAL

The SHUB engine writes the collected sensor data during hibernation to a non-volatile external flash memory. The external flash memory contains a dedicated memory allocation to write this data into, as specified by the value in USER_SIZE mentioned earlier. On wakeup after hibernation, the SoC reads this data from the external flash memory and transmits this data through the primary radio.

ATM33/e Series

In the ATM33/e series, with the absence of an external flash memory, the data is stored on the onboard volatile SRAM memory. This memory size is allocated in the memory map section of the SRAM. This could be done in any of the linker scripts depending on the compiler used (such as GCC, ARM or IAR compilers). For example, if using the GCC compiler, this section exists in the nonsecure_sections.ld file.

To ensure that this SRAM memory gets its own allocation, a special section in memory called the `uninit` section is defined in the linker descriptor files (in case of GCC as a compiler) or in the scatter files (in case of ARM compiler) during compile time. This is done to ensure the data collected and stored by the SHUB engine does not get overwritten during hibernate and the SHUB engine always has access to a unique memory location which is known to contain the data collected.

```
.uninit (NOLOAD) :
        {
                . = ALIGN(4);
                *(SORT_BY_ALIGNMENT(.uninit*))
                . = ALIGN(4);
        } > RAM
```

Figure 15 - Allocation of Memory for ATM33/e

It is important to note that the `uninit` memory section is defined in the Non-secure memory region of the application.

Since the SRAM memory is volatile, when the SOC goes into hibernation mode, this SRAM section might also get powered off like the other sections. To avoid this, the SHUB engine must configure the Power Sequencer (PSEQ) registers to OVERRIDE certain SRAM memory banks where the SHUB data is expected to be stored.

CONFIDENTIAL

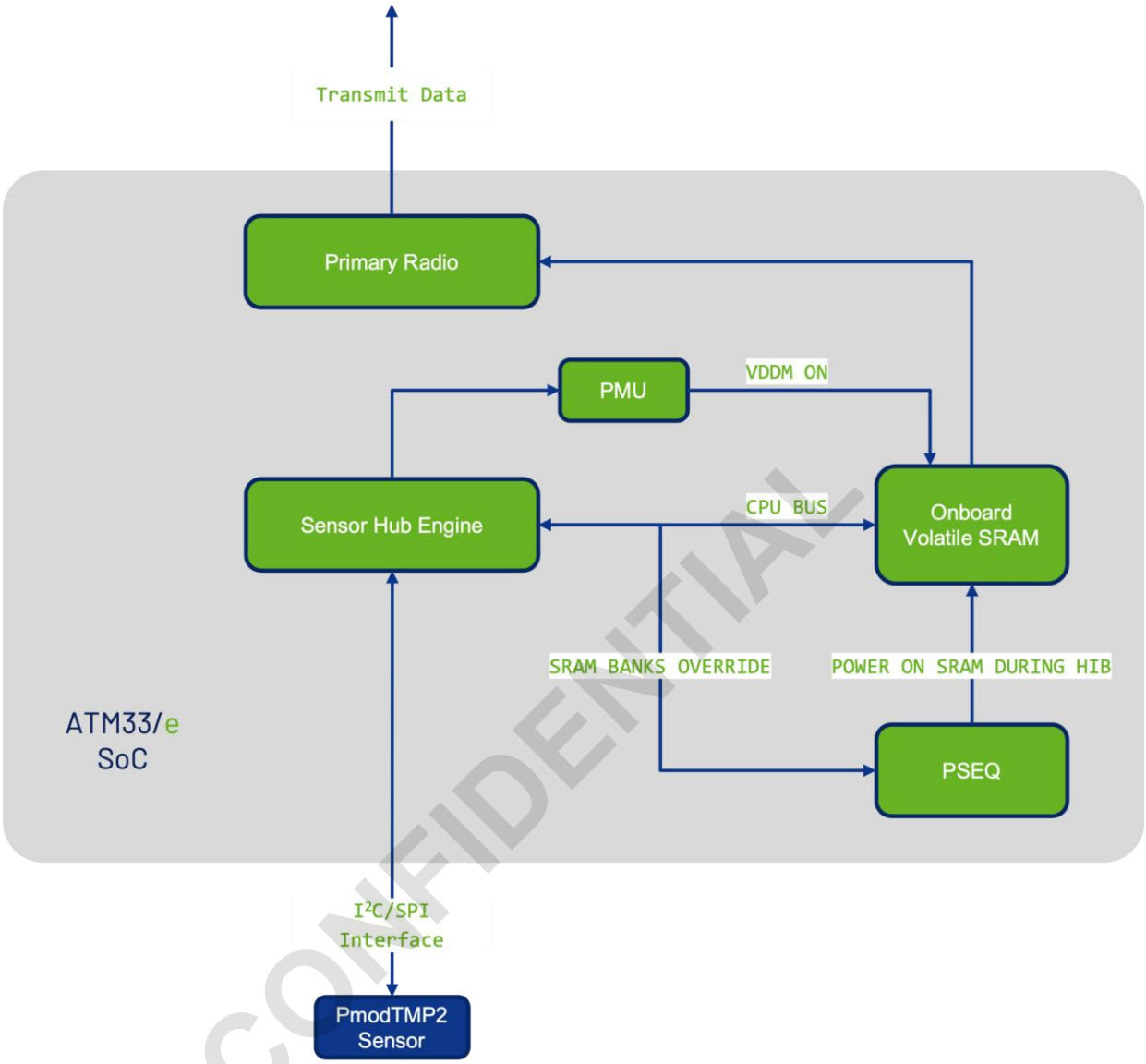
```
// enable sensor hub functionality
        CMSDK_PSEQ->SENSOR_HUB_CONTROL =
            PSEQ_SENSOR_HUB_CONTROL__SHUB_SRAM0_ACC_EN__WRITE(1) |
            PSEQ_SENSOR_HUB_CONTROL__SHUB_SRAM1_ACC_EN__WRITE(1) |
            PSEQ_SENSOR_HUB_CONTROL__SHUB_SRAM2_ACC_EN__WRITE(1) |
            PSEQ_SENSOR_HUB_CONTROL__SHUB_SRAM3_ACC_EN__WRITE(1) |
            PSEQ_SENSOR_HUB_CONTROL__SHUB_VDDCUT__WRITE(0) |
            PSEQ_SENSOR_HUB_CONTROL__SHUB_ISO__WRITE(0) |
            PSEQ_SENSOR_HUB_CONTROL__SHUB_CLKEN_SLOW__WRITE(1) |
            PSEQ_SENSOR_HUB_CONTROL__SHUB_CLKEN__WRITE(1) |
            PSEQ_SENSOR_HUB_CONTROL__SHUB_START_ADV_OVRD__WRITE(1) |
            PSEQ_SENSOR_HUB_CONTROL__SHUB_START_ADV_VAL__WRITE(0) |
            PSEQ_SENSOR_HUB_CONTROL__SHUB_RUNNING_OVRD__WRITE(0) |
            PSEQ_SENSOR_HUB_CONTROL__SHUB_RUNNING_VAL__WRITE(0) |
            PSEQ_SENSOR_HUB_CONTROL__SHUB_RESET_SLOW__WRITE(1) |
            PSEQ_SENSOR_HUB_CONTROL__SHUB_RESET__WRITE(1) |
            PSEQ_SENSOR_HUB_CONTROL__ENABLE_FUNC__WRITE(1);


        PSEQ_SENSOR_HUB_CONTROL__SHUB_RESET__MODIFY(CMSDK_PSEQ
                                                    -
>SENSOR_HUB_CONTROL,
            0);
        PSEQ_SENSOR_HUB_CONTROL__SHUB_RESET_SLOW__MODIFY(
            CMSDK_PSEQ->SENSOR_HUB_CONTROL, 0);
```

Figure 16 - SHUB OVERRIDE Power Configuration

CONFIDENTIAL

Figure 17 shows how the SHUB engine interacts with the PSEQ and the PMU to keep the SRAM banks powered on during hibernation.



CONFIDENTIAL

Figure 17 - Memory Layout and Access for ATM33/e

After waking up from hibernation, data is read from the SRAM banks and is transmitted via the primary radio.

Reference Documents

Title	Document Number
EVK User's Guide for ATMx301/ATMx202	ATMx201-UG
EVK User's Guide for ATMx221	ATMx221-UG
ATM33/e Series Evaluation Kit User Guide	ATM33_e-UGEVK
HT_thermometer Example Application Note (regarding ADT7240 sensor)	ATMx2xx-ANHTE
Using ATM2/ATM3 Sensor Hub White Paper	ATMxxxx-WPSH

CONFIDENTIAL

Revision History

Date	Version	Description
April 25, 2023	0.50	Initial version created.

CONFIDENTIAL

ATMOSIC TECHNOLOGIES – DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2023 by Atmosic Technologies. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc. All other trademarks are the properties of their respective holders.

CONFIDENTIAL

Atmosic™

Atmosic Technologies | 2105 S. Bascom Ave. | Campbell CA, 95008
www.atmosic.com